Apartment Commute Optimizer

Final Report

CS5800 Spring 2026 Group Project

Xujing Hui (#002339163)

Fanchao Yu (#001601606)

Jiaxin Liu (#003169274)

1. Introduction

1.1 Background and Motivation

For graduate students enrolled at Northeastern University Silicon Valley, finding suitable housing is one of the first and most consequential decisions of their academic journey. The San Jose metropolitan area spans multiple cities: Sunnyvale, Santa Clara, Milpitas, and downtown San Jose, each offering different apartment options at different price points. However, rent alone does not capture the full cost of an apartment: a cheaper unit that is far from campus, far from a gym, or disconnected from grocery transit could cost a student dozens of additional commute hours every semester.

Most apartment-comparison tools focus on proximity to a single destination, typically the workplace or school, measured in straight-line distance. This approach fails students who depend on public transit and who must regularly visit multiple categories of destinations. A student commuting to NEU three times per week, visiting a Starbucks on the way each time, going to the gym four times per week, and running grocery errands twice weekly accumulates a significant transit burden that cannot be reduced to a single map measurement.

1.2 Research Question

Among a set of candidate apartments in the San Jose metropolitan area, which feasible apartment minimizes total weekly transit commute time to a student's fixed set of destinations: school, gym, groceries, and daily coffee, subject to rent and transit-accessibility constraints?

1.3 Personal Investment: Why This Topic Matters to Us

Xujing Hui

As an international student who recently went through the apartment search process in San Jose, I found the experience overwhelming. I compared dozens of listings and ultimately chose based on rent and gut feeling about the neighborhood, without any systematic analysis of how my commute patterns would actually play out. After the first month, I realized I was spending over an hour every day just on gym and grocery trips combined. This project is my attempt to build the tool I wish I had before signing my lease. The Dijkstra component of this project is also directly connected to what I find most intellectually compelling in CS5800 — the elegance of turning a messy real-world decision into a clean graph problem with a provably optimal solution.

Fanchao Yu

I came to this project from a data visualization perspective. I have always been interested in how abstract algorithm outputs can be made legible and actionable to someone who is not a computer scientist. The apartment optimizer is a perfect case study: the results of Dijkstra's algorithm are ultimately meaningless unless a prospective tenant can understand why one apartment scored better than another. My personal investment is in the communication layer — building the Kepler.gl visualization and the final presentation so that the algorithm's decisions are transparent and explainable. I also live in the Santa Clara area and have direct experience navigating the VTA bus network, which informed many of our data-collection decisions about which routes and stops to include.

Jiaxin Liu

Before this project, I spent three years as an analyst at a property management company managing units near USC in Los Angeles. When pricing apartments, we almost always reduced location value to a single

metric: distance to campus, which always felt inaccurate. This project gave me a framework to think about that problem more rigorously. By modeling a full transit network and weighting destinations by actual visit frequency, the score reflects how a neighborhood actually functions for a specific lifestyle, not just how close it is to one anchor point.

My technical contributions were on the data and modeling side. I sourced and structured the transit and destination data, built the visualizations, and proposed two modeling decisions that came directly from my own commute habits. The first was the chain-trip model, which scores the Starbucks-to-NEU leg as a single linked trip rather than two independent destinations. The second was the walk-fallback edges, which account for cases where walking is genuinely faster than waiting for a bus.

2. Technical Discussion and Analysis

2.1 GenAI Interaction and How We Made Our Solution Unique

We began by querying both ChatGPT (GPT-4) and Claude with the following prompt:

"Given five apartments near a university campus, which one is the best choice for a student based on commute?"

Both tools responded with suggestions to calculate straight-line distance between each apartment and the university, rank by distance, and optionally look up walking time using the Google Maps API. Neither tool framed the problem as a graph optimization. Neither tool suggested modeling multiple destination categories, nor weighting by visit frequency. This initial response confirmed that GenAI would produce an oversimplified, single-destination answer.

We refined our prompt iteratively:

1. Added transit graph constraint: "Consider that the student uses public transit and must route through bus and Caltrain stops." GenAI suggested a distance matrix but still pre-assigned one store per category (e.g., 'pick the nearest Costco') rather than letting the algorithm discover the optimal store.
2. Added multi-destination and visit-frequency weighting: "Score each apartment by sum of weighted travel times across multiple destination categories." GenAI produced pseudocode with a for-loop but used straight-line distances instead of graph-based shortest paths.
3. Added chain trip: "The student always stops at Starbucks before going to school." GenAI could not model the two-leg chain trip. It continued scoring Starbucks and school independently, which underestimates the actual trip cost.
4. Added walk-fallback edges: "If two nodes are less than 30 minutes' walk apart but have no transit path, add a direct edge." GenAI acknowledged this was a valid graph construction choice but could not identify which specific pairs in our dataset required it.

The final version of our algorithm includes all 4 improvements that GenAI could not provide without prompt: (1) Dijkstra on a real multi-modal transit graph, (2) letting the algorithm select the best store for each category rather than pre-assigning one, (3) modeling the Starbucks and NEU visit as a single linked trip rather than two separate destinations, and (4) adding walk-fallback edges derived from Haversine distance for cases where walking is faster than transit. These improvements produced a ranking that differs from GenAI's output in both the winner (Villas on the Boulevard, score 222.2 min/week) and the ordering of ranks 3 and 4.

Dimension	GenAI Approach	Our Algorithm
Routing model	Straight-line or Google Maps API	Dijkstra on 66-node multi-modal graph
Store selection	Human pre-selects nearest store	Algorithm discovers optimal via graph search
School commute	NEU scored independently	Chain trip: Apt → Starbucks → NEU
Walk handling	Ignored or 15-min blanket assumption	Haversine fallback edges, cap = 30 min

Dimension	GenAI Approach	Our Algorithm
NEU computation	Repeated per apartment (30× waste)	Precomputed once; reused via undirected property
Transit network	Single-mode or approximate	Caltrain + VTA Rapid/Local bus + BART

2.2 Data Collection and Input Design

All data was collected manually and stored in three JSON files. No automated scraping was used. Sources and methodology for each file are described below.

File	Contents	Source	Date
apartments_json_v2.json	5 apartments: address, lat/lng, monthly rent, area, nearest_station_id, walk_min_to_station, transit accessibility note.	Zillow, Apartments.com, Google Maps	April 2026
destinations_json_v2.json	25 destinations across 5 categories: address, lat/lng, nearest_station_id, walk_min_from_station, weekly_visits	Google Map	April 2026
stations_json_v2.json	36 transit nodes: Caltrain stations, VTA bus hubs, VTA bus stops, Milpitas BART, each with a neighbors array listing adjacent nodes and travel_min	Caltrain schedule Jan 2026, VTA route map 2025, Google Map, Moovit	April 2026

Apartment coordinates were verified against Google Maps street view to ensure the pin was placed at the building entrance. Walk times from apartments and destinations to their nearest station were measured using Google Maps walking directions. Transit travel times (travel_min on station-to-station edges) use in-vehicle time only, and waiting/headway time is excluded as a known limitation.

The five apartments were selected to span geographically distinct areas of Santa Clara County so that Dijkstra would produce meaningfully different shortest paths for each. Avoiding the similar results we obtained in our initial prototype that placed all apartments along a single transit line.

2.3 Algorithm Design and CS5800 Course Connections

The problem maps directly onto the single-source shortest-path problem, a core topic from CS5800. The graph G = (V, E) is weighted, undirected, and has all non-negative edge weights, which is the prerequisite for using Dijkstra's algorithm. Nodes represent apartments, destinations, and transit stations (V = 66). Edges represent traversable routes weighted by travel time in minutes ($|E| \approx 94$). The objective is to minimize the weighted sum of shortest-path travel times to all destination categories, using one Dijkstra run per apartment source node.

Graph Construction

The graph is built in four steps corresponding to four edge types. The full pseudocode appears in the Appendix (pseudocode_1_build_graph.txt in the submitted ZIP).

```
function BUILD_GRAPH(apartments, destinations, stations):
  G ← empty adjacency list
  // Step 1: station ↔ station (schedule time from JSON neighbors)
  for each station s: for neighbor n: ADD_EDGE(G, s.id, n.id,
n.travel_min)
  // Step 2: apartment → nearest station (walk_min_to_station)
  for each apartment a: ADD_EDGE(G, a.id, a.nearest_station_id, a.walk)
  // Step 3: destination → nearest station (walk_min_from_station)
  for each destination d: ADD_EDGE(G, d.id, d.nearest_station_id, d.walk)
  // Step 4: walk-fallback — add direct edge if haversine walk ≤ 30 min
  for each (apt a, dest d):
    w = haversine(a,d) × 1.3 / 80    // 80 m/min walk speed, 1.3 detour
    if w ≤ 30: ADD_EDGE(G, a.id, d.id, w)
  return G
```

The walk-fallback step (Step 4) is a design contribution by Jiaxin Liu. It addresses cases where two nodes are geographically close but no transit route connects them efficiently. For example, the apartment Villas on the Boulevard and the gym G2 (24 Hour Fitness Santa Clara ECR) are 350 meters apart. Without a fallback edge, Dijkstra routes through BUS_ECR_SCOTT → BUS_ECR_LAWRENCE → G2, a total of 13 minutes. With the fallback edge, Dijkstra selects the 4.6-minute direct walk. This reduces Villas's gym contribution from 4×13 = 52.0 to 4×4.6 = 18.4 minutes per week, a 33.6 min/week improvement that is the primary driver of its first-place ranking. Nine such fallback edges were activated in our dataset.

Dijkstra's Algorithm (Primary Solver)

```
function DIJKSTRA(G, source):
  dist ← {all nodes: ∞};   dist[source] ← 0
  prev ← {all nodes: None}
  heap ← min-heap;   PUSH(heap, (0, source));   visited ← ∅
  while heap ≠ ∅:
    (d, u) ← POP_MIN(heap)
    if u ∈ visited: continue           // lazy deletion
    visited ← visited ∪ {u}
    for each (v, w) in G[u]:
      if d + w < dist[v]:
        dist[v] ← d + w;   prev[v] ← u
        PUSH(heap, (dist[v], v))
  return dist, prev
```

The lazy-deletion pattern is used because Python's heapq module does not support a decreaseKey operation. A stale (distance, node) entry that is popped after the node has already been settled is simply skipped. This adds at most one extra heap entry per edge relaxation, giving a total heap size of O(E) in the worst case, which preserves the O((V + E) log V) time bound.

Bellman-Ford is implemented in dijkstra.py as a cross-validation tool. Because all edge weights are non-negative, Dijkstra is both correct and strictly faster. Bellman-Ford runs in O(V × E) time and was used during development to verify that Dijkstra produced identical distance maps for all 5 apartment sources.

Chain Trip and NEU Precomputation (Jiaxin Liu)

```
// Precompute from NEU once (runs outside the apartment loop)
```

```
dist_neu, _ ← DIJKSTRA(G, 'NEU_SV')

// Inside the apartment scoring loop:
chain_cost ← ∞;   best_sb ← None
for each Starbucks_k in {S1, S2, S3, S4, S5, S6}:
  d_apt_sb ← dist_apt[Starbucks_k]
  d_sb_neu ← dist_neu[Starbucks_k]   // free from precompute
  if d_apt_sb + d_sb_neu < chain_cost:
    chain_cost ← d_apt_sb + d_sb_neu
    best_sb    ← Starbucks_k
```

This design replaces the original model where Starbucks and NEU were scored as two independent destinations. The original formulation was equivalent to assuming the student makes separate trips to Starbucks and to NEU, which is inaccurate. The chain trip correctly captures that each school visit is a single journey: Apartment → Starbucks* → NEU. The selected Starbucks S* is not always the one closest to the apartment — it is the one that minimizes the sum of both legs. For Cannery Park, S5 (Downtown SJ, 19 min from apartment) beats S3 (41 min from apartment) because S5 is only 28 minutes from NEU, while S3 is 34 minutes from NEU.

The NEU precomputation is an efficiency improvement. Because G is undirected, the shortest path from Starbucks_k to NEU equals the shortest path from NEU to Starbucks_k. Running one Dijkstra from NEU gives us all six dist_neu[Sk] values simultaneously, eliminating what would otherwise be 5 × 6 = 30 redundant sub-computations.

Full Optimizer Pipeline

```
function FIND_BEST_APARTMENT(apartments, destinations, stations,
constraints):
  G        ← BUILD_GRAPH(apartments, destinations, stations)
  dist_neu ← DIJKSTRA(G, 'NEU_SV')                    // precompute once
  results  ← []
  for each apartment a:
    if a.rent > constraints.max_rent: skip
    if a.walk > constraints.max_walk: skip
    dist_apt, prev ← DIJKSTRA(G, a.id)                // one run per apt
    chain  ← min_k [ dist_apt[Sk] + dist_neu[Sk] ]    // chain trip
    costco ← min_k [ dist_apt[Ck] ]                   // best of 6
    tj     ← min_k [ dist_apt[Tk] ]
    gym    ← min_k [ dist_apt[Gk] ]
    score  ← 3×chain + 1×costco + 2×tj + 4×gym        // weighted sum
    APPEND (a, score, breakdown) to results
  SORT results by score ascending
  return results
```

2.4 Time and Space Complexity

Component	Time Complexity	Space Complexity	Notes
Build graph (Steps 1–3)	O(V + E)	O(V + E)	One pass through neighbor arrays
Walk-fallback check (Step 4)	O(A × D)	O(1)	A=5 apts, D=25 dests = 125 checks

Component	Time Complexity	Space Complexity	Notes
NEU precompute (1 Dijkstra)	O((V+E) log V)	O(V)	Runs once outside apartment loop
Dijkstra per apartment	O((V+E) log V)	O(V)	5 runs total, V=66, E≈94
Full pipeline (A apartments)	O(A · (V+E) log V)	O(V+E)	Dominated by Dijkstra runs
Bellman-Ford (verification)	O(V × E)	O(V)	Used in testing only
Category scoring	O(A × 6)	O(1)	Linear scan over 6 candidates each
Sorting	O(A log A)	O(A)	A=5, negligible

The graph contains V = 66 nodes: 5 apartment nodes, 25 destination nodes, and 36 transit nodes (7 Caltrain stations, 3 VTA bus hubs, 21 VTA bus stops, and 1 Milpitas BART station). The edge count E ≈ 94 was obtained by running graph_stats() after graph construction, which counts each undirected edge once. A = 5 is the number of candidate apartments entering the scoring loop after constraint filtering, and D = 25 is the total number of destination nodes across all categories.

For our concrete dataset, a single Dijkstra run processes approximately $(V + E) \times \log_2 V = 160 \times 6 \approx 960$ heap operations. The full pipeline runs Dijkstra 6 times (5 apartment sources + 1 NEU precompute), for a total of roughly 5,760 operations — completing in under 10 milliseconds on any modern hardware. This confirms that Dijkstra is appropriate for our problem scale. Even scaling to hundreds of apartments and thousands of nodes, the O((V + E) log V) per-source complexity would remain tractable.

2.5 Library Functions Used

Function	Python Module	Purpose	Reference
heapq.heappush / heapq.heappop	heapq (stdlib)	Binary min-heap for Dijkstra's priority queue. Provides O(log n) insert and extract-min, enabling the O((V+E) log V) time bound.	Python Docs: docs.python.org/3/library/heapq.html
math.radians, math.sin, math.cos, math.asin, math.sqrt	math (stdlib)	Trigonometric functions used in the Haversine formula to compute great-circle distance between two lat/lng coordinates.	Python Docs: docs.python.org/3/library/math.html
json.load	json (stdlib)	Parse the three JSON input files (apartments, destinations, stations) into Python dicts and lists.	Python Docs: docs.python.org/3/library/json.html

No third-party (pip-installable) packages are required. The entire project runs on the Python 3.10+ standard library.

2.6 Iterative Development — v1 to v2

Iteration 1 (v1): Single Transit Corridor, Fixed Destinations

Our initial prototype (Report 1) modeled 5 apartments along the VTA Blue Line (a single linear corridor), with 3 fixed destinations: NEU, a Walmart, and Planet Fitness. The transit graph had 8 stations and 3-minute uniform edges. Dijkstra ran correctly but had no branching decisions to make. The graph was a linear chain, so the algorithm simply counted hops. Starbucks and NEU were scored independently. No walk-fallback edges existed. Results showed tiny score differences between apartments driven entirely by position along one axis.

Problems identified in v1:

- Transit graph was a trivial linear chain: Dijkstra provided no routing insight.
- Destinations were manually pre-assigned (e.g., 'this apartment uses that Walmart'). The algorithm's core value, finding minimum-cost paths among multiple options, was bypassed.
- All 5 apartments were in the same geographic corridor, making score differences negligible.
- Starbucks and NEU scored as two independent trips, which provided an incorrect behavioral model.

Iteration 2 (v2): Multi-Modal Graph, Chain Trip, Walk Fallback

In v2 (current version), we rebuilt the graph as a 66-node multi-modal network covering Caltrain, VTA Rapid 500/522, VTA local bus routes 20/22/57/72/73, and Milpitas BART. Destinations were expanded to 25 nodes across 5 categories, each with 6 candidate locations. The algorithm now discovers the optimal store per category without human pre-assignment. Two additional improvements were proposed by Jiaxin Liu:

- Chain trip model: Starbucks and NEU combined into a single two-leg trip, scored as 3 × min_k [d(Apt,Sk) + d(Sk,NEU)].
- Walk-fallback edges: For any (apartment, destination) pair with Haversine walking time ≤ 30 minutes, a direct walk edge is added to the graph, allowing Dijkstra to choose walking over transit when it is faster.

Impact of v2 changes:

Apartment	v1 Score	v2 Score	Change	Driver
Villas on the Boulevard	269.0	222.2	−46.8	Walk-fallback: gym 13→4.6 min, SB 12→4.6 min
Murphy Station	302.0	238.1	−63.9	Walk-fallback: TJ 20→6.7 min, SB 19→6.1 min
The Verdant	330.0	272.5	−57.5	Walk-fallback: SB S4 = 1.7 min (nearly co-located)
Cannery Park	309.0	294.0	−15.0	Chain trip: S5 Downtown SJ saves SB→NEU leg
The Harlowe	346.0	338.4	−7.6	No walk-fallback pairs; chain trip savings minor

The ranking also changed between v1 and v2: The Verdant rose from #4 to #3, and Cannery Park dropped from #3 to #4. This shift is driven by The Verdant's co-located Starbucks S4 (1.7-minute walk), which gives it the best chain-trip score of any apartment (36.7 minutes per trip × 3 visits = 110.1 min/week) - an advantage invisible to the v1 model.

2.7 Testing - Inputs, Outputs, and Discussion

We implemented 22 unit tests across three test files. All 22 tests pass. Tests are organized into three modules corresponding to the three source files.

Test File	Tests	What Is Tested
test_graph.py	6	Node count = 66; bidirectionality of all edges; presence of walk-fallback edges for 5 known pairs; fallback cap ≤ 30 min; apartments connected to stations; destinations connected to stations
test_dijkstra.py	8	Source distance = 0; unreachable node = infinity; known hand-computed distances on small graph; Dijkstra = Bellman-Ford on real graph (5 sources); path reconstruction correctness; source-to-self path; unreachable path returns None; NEU distances in [5, 90] min for all apartments
test_optimizer.py	8	All 5 apartments pass constraints; result count = 5; winner = villas_on_the_blvd; last = the_harlowe; scores strictly ascending; expected scores within ±0.5 min tolerance; correct Starbucks winner per apartment; Villas gym uses walk-fallback (<10 min)

Selected test inputs and outputs with discussion:

Test: Dijkstra vs. Bellman-Ford cross-validation

Input: Real transit graph G (66 nodes, ~94 edges); each of the 5 apartments as source. Output: dist_dijkstra == dist_bellman_ford for all (source, node) pairs within tolerance 1e-6. This test confirmed that our lazy-deletion Dijkstra implementation is correct and that no floating-point accumulation errors affect results.

Test: Walk-fallback cap respected

Input: Graph G after build_graph(). Output: no (apt, dest) edge with weight > 30 minutes. This test guards against accidentally adding long-distance walk edges that would give unrealistically low scores to apartments that are far from destinations as the crow flies but far by road.

Test: Chain trip Starbucks winners

Input: run_optimizer() on real data. Output: verified that each apartment selects the expected Starbucks — notably, Cannery Park selects S5 (Downtown SJ, 47 min chain) over S3 (Santa Clara, 75 min chain), despite S3 being geographically closer. This result is surprising at first glance but is correct: S5 is only 28 minutes from NEU via walking, making the second leg dramatically shorter.

Suggested improvements to testing:

- Add real-time data injection tests using mock GTFS feeds to verify behavior when transit schedules change.
- Add a stress test with a large synthetic graph (1,000+ nodes) to verify algorithmic scalability.
- Add tests for edge cases: apartment that IS a destination node; two apartments at identical locations.

2.8 Results

The objective function Score(a) = 3×chain + 1×Costco + 2×TJ + 4×Gym produces the following ranking:

Rank	Apartment	Area	Rent	Chain×3	Costco×1	TJ×2	Gym×4	Total (min/wk)
#1	Villas on the Boulevard	Santa Clara	$3,465	135.6	29.0	39.2	18.4	222.2
#2	Murphy Station	Sunnyvale	$3,817	126.3	18.0	13.4	80.4	238.1
#3	The Verdant Apartments	North SJ	$3,292	110.1	23.0	67.4	72.0	272.5
#4	Cannery Park by Windsor	Downtown SJ	$3,359	141.0	29.0	36.0	88.0	294.0
#5	The Harlowe	Milpitas	$3,300	195.0	45.0	26.4	72.0	338.4

Walk-fallback edges activated (confirmed by test_optimizer.py): Villas ↔ S2 (4.6 min), Villas ↔ G2 (4.6 min), Murphy ↔ S1 (6.1 min), Murphy ↔ T1 (6.7 min), Verdant ↔ S4 (1.7 min), Cannery ↔ NEU_SV (29.9 min), Cannery ↔ T2 (20.1 min), Harlowe ↔ T3 (13.2 min), Harlowe ↔ S6 (20.1 min).

An interactive visualization of all transit paths was built by Jiaxin Liu using Kepler.gl (15 layers: 10 point layers + 5 per-apartment GeoJSON path layers). Live map: https://kepler.gl/demo/map?mapUrl=https://dl.dropboxusercontent.com/scl/fi/201mm3bxqt38qckr7xfgj/keplergl_mx1tfv.json?rlkey=jecgfeam9onwax72sg2snfcdk&dl=0

3. Conclusion

3.1 Answer to the Research Question

Among the five candidate apartments, Villas on the Boulevard (Santa Clara, $3,465/month) is the optimal choice with a total weekly commute score of 222.2 minutes. Its first-place ranking is driven by walk-accessible Starbucks S2 and Gym G2 (both 4.6 minutes on foot), which produce the lowest gym contribution of any apartment: 4 × 4.6 = 18.4 min/week, compared to Murphy Station's 80.4 min/week. Over a 40-week academic year, choosing Villas over Murphy Station saves approximately 13 hours of commute time. Choosing Villas over last-place The Harlowe saves approximately 77 hours annually, more than three full working days.

The chain-trip model revealed a non-obvious result: The Verdant Apartments (#3) has the best school-commute chain of any apartment (36.7 min/trip × 3 = 110.1 min/week), because its co-located Starbucks S4 costs only 1.7 minutes to reach on foot. Without the chain-trip model, this advantage was invisible in v1, causing The Verdant to rank #4. This demonstrates that accurately modeling behavioral patterns, not just geographic proximity, is essential for the optimizer to produce actionable results.

3.2 Weaknesses and Limitations

- Travel times are in-vehicle minutes only. Waiting time (VTA buses average 15–30 min headway) is excluded. In practice, actual door-to-door times are 10–30 minutes longer than our scores indicate.
- Walk-fallback edges use Haversine × 1.3 (straight-line plus 30% detour) [11]. Actual pedestrian routes may differ significantly in areas with highways, private property, or elevation changes.
- Static schedule data only. The model uses Caltrain schedules from January 2026 and VTA maps from 2025. Service disruptions, weekend reduced frequency, and holiday schedules are not captured.
- The chain-trip model assumes the student always visits Starbucks before NEU on every school day. On days when Starbucks is skipped, the actual cost is just d(apt, NEU).
- Rent is point-in-time (March 2026). Utilities, parking fees, deposits, and lease-term flexibility are excluded from the comparison.

3.3 Avenues for Future Research

- Integrate GTFS real-time feeds to model headway waiting time as a probabilistic variable, producing an expected total-trip time rather than in-vehicle time only.
- Generalize the chain-trip framework to arbitrary ordered multi-stop sequences: Apt → Stop1 → Stop2 → … → Destination, enabling optimization of complex daily routines.
- Add a Pareto-front analysis visualizing the rent vs. commute-score trade-off, allowing users to select their preferred balance rather than accepting a single winner.
- Implement A* search with a geographic heuristic (straight-line distance to destination) to reduce the number of nodes explored, improving scalability for larger city-scale graphs.
- Build a web UI where students enter their own weekly visit frequencies and see the ranking update in real time, personalizing the optimization to their lifestyle.

3.4 Individual Reflections

Xujing Hui

Before this course, I chose my apartment based on rent and gut feeling. After moving in, I realized my weekly gym and grocery trips were costing me more commute time than getting to campus. That frustration is what this project tries to solve.

My primary contribution was the algorithm layer: implementing Dijkstra with a min-heap and Bellman-Ford in dijkstra.py, designing the overall system architecture, writing the pseudocode and complexity analysis, and building the test suite that validates correctness across all three modules. On the technical side, the hardest part was not writing Dijkstra itself — the textbook pseudocode is clear, and translating it into Python with heapq took less than an hour. The real difficulty was everything around it. When I first ran the algorithm on our 66-node graph, some apartments returned infinity for destinations that were clearly reachable on a map. It took two hours of debugging to find the cause: a destination node disconnected from all stations because its walk time was 15.3 minutes, just over our 15minute cap. The fix was one line, but the experience taught me that graph construction errors are much harder to catch than algorithm errors, because the algorithm runs correctly on the wrong graph.The other thing I did not expect was how much the objective function matters. In our first version, we scored Starbucks and NEU as two separate destinations, and the results felt wrong. Jiaxin pointed out that in real life you stop at Starbucks on the way to school, not as a separate trip. Once we changed the model to a chain trip, The Verdant moved from fourth to third place because it has a Starbucks practically next door. The algorithm did not change at all, only the problem formulation changed. That was probably the biggest lesson: Dijkstra always gives you the optimal answer to whatever you ask, so you need to make sure you are asking the right question.

Fanchao Yu

My primary takeaway from this project is that algorithm output is only as useful as its presentation. I spent as much time on the Kepler.gl visualization and the presentation structure as on the code itself, and I believe that investment was justified. A result that cannot be communicated clearly might as well not exist. From a technical standpoint, the most valuable lesson was the chain-trip insight: the objective function design matters as much as the algorithm choice. Dijkstra on a poorly-structured objective function will produce a correct solution to the wrong problem. I will apply this lesson — model the problem carefully before choosing the algorithm — in every data project I undertake after graduation.

Jiaxin Liu

The two modeling improvements I contributed — chain trips and walk-fallback edges — came from a simple question I asked myself while reviewing the v1 results: does this actually describe how I live? When the answer was no, I looked for the minimal change to the graph that would make it say yes. This behavior-first approach to graph construction is something I had not seen explicitly taught in a course before, and it is a method I intend to carry into future work.

What struck me most was how naturally this framework connected to my prior experience in property management. For three years, I watched our team price apartments using distance to campus as a proxy for everything, which always felt like collapsing a multidimensional reality into a single number. This project showed me that a transit-weighted scoring model is not significantly harder to build than a distance ranking, yet it captures something far more accurate about how a neighborhood actually functions for a specific person. I think there is a real opportunity for a framework like this to influence how the real estate industry thinks about location value, shifting from a static geographic measurement toward something lifestyle-dependent and quantifiable.

4. Assumptions and Limitations

Assumptions

- One-way travel time only. We compute all scores using one-way trips. The return trip is assumed to take the same time because the graph is undirected.
- Fixed weekly visit counts. We use fixed numbers: NEU+Starbucks 3×, Gym 4×, Trader Joe's 2×, Costco 1×. A student with different habits would get a different ranking.
- Walk-fallback cap = 30 minutes. If an apartment and a destination are within 30 minutes' walk, we add a direct walking edge. 30 minutes is the longest walk a typical person would consider. Changing this threshold would change which edges exist and could change the results.
- Undirected graph. Every edge has the same weight in both directions. This is accurate for Caltrain and walking, but some VTA bus routes may be slightly faster in one direction due to one-way streets.

Limitations

- No waiting time. Our edge weights only include time spent walking or riding. They do not include time spent waiting for a bus. VTA buses run every 15–30 minutes during off-peak hours, so real door-to-door times are longer than our scores suggest.
- Static schedules. We use Caltrain schedules from January 2026 and VTA route maps from 2025. Weekend service, holiday schedules, and service disruptions are not captured.
- Approximate walking distances. Walking time is estimated using straight-line distance × 1.3 (to account for streets not being straight). Actual walking routes may differ due to highways, fences, or hills.
- Small candidate set. We evaluate 5 apartments, not the entire rental market. The winner is the best of these 5, not necessarily the best apartment in San Jose.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022. — Dijkstra's algorithm (Ch. 22) and graph representation (Ch. 20).

[2] E. W. Dijkstra, 'A note on two problems in connexion with graphs,' Numerische Mathematik, vol. 1, pp. 269–271, 1959.

[3] Python Software Foundation, 'heapq — Heap queue algorithm,' Python 3.12 Documentation. [Online]. Available: https://docs.python.org/3/library/heapq.html

[4] Python Software Foundation, 'math — Mathematical functions,' Python 3.12 Documentation. [Online]. Available: https://docs.python.org/3/library/math.html

[5] Caltrain, 'Caltrain Weekday Timetable,' effective January 31 2026. [Online]. Available: https://www.caltrain.com/schedules

[6] Santa Clara Valley Transportation Authority, 'VTA System Map and Route Guide,' 2025. [Online]. Available: https://www.vta.org/go/routes

[7] Kepler.gl, 'Kepler.gl — Open-source geospatial analysis tool,' Uber Engineering. [Online]. Available: https://kepler.gl

[8] W. G. Van Pelt, 'Haversine formula,' in Encyclopedia of Mathematics. Springer, 2001. Implemented via Python math module.

[9] Zillow Group, 'Zillow Rental Listings,' March 2026. [Online]. Available: https://www.zillow.com

[10] Google LLC, 'Google Maps Platform — Directions API,' accessed March 2026. [Online]. Available: https://developers.google.com/maps

[11] D. J. Giacomin and D. M. Levinson, "Road network circuity in metropolitan areas," Environment and Planning B: Planning and Design, vol. 42, no. 6, pp. 1040–1053, 2015.

Appendix — Source Code and Pseudocode

All source code and pseudocode files are included in the submitted ZIP file ReportHuiLiuYu.zip. The directory structure is as follows:

```
ReportHuiLiuYu.zip
├── data/
│   ├── apartments_json.json        5 candidate apartments
│   ├── destinations_json.json      25 destinations (5 categories)
│   └── stations_json_v2.json       36 transit nodes + neighbor edges
├── src/
│   ├── utils.py                    Haversine distance, walk-time helpers
│   ├── graph.py                    Graph builder (load JSON + build
edges)
│   ├── dijkstra.py                 Dijkstra, Bellman-Ford, path
reconstruction
│   ├── optimizer.py                Main pipeline: filter → score → rank
→ output
│   ├── test_graph.py               6 unit tests for graph construction
│   ├── test_dijkstra.py            8 unit tests for shortest-path
algorithms
│   └── test_optimizer.py           8 end-to-end optimizer tests
├── pseudocode/
│   ├── pseudocode_1_build_graph.txt
│   ├── pseudocode_2_dijkstra.txt
│   └── pseudocode_3_optimizer.txt
├── results/
│   ├── ranking_output.txt          Full ranked results with score
breakdown
│   └── test_output.txt             22/22 tests passed
└── README.md                       Setup and run instructions
```

To run the optimizer:

```
cd src
python optimizer.py
```

To run all 22 tests:

```
cd src
python test_graph.py
python test_dijkstra.py
python test_optimizer.py
```

Requirements: Python 3.10 or later. No third-party packages required (standard library only: heapq, math, json, os, sys).